

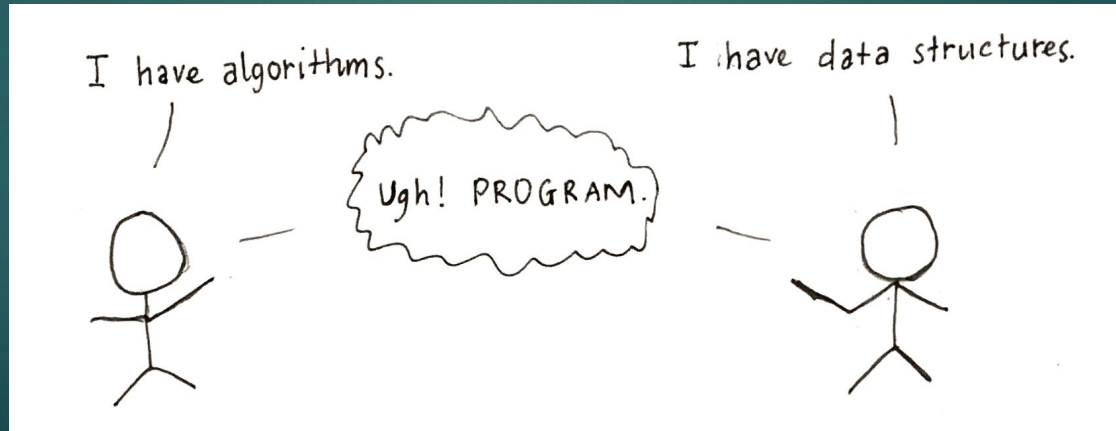


AMCS2034 Introduction to Data Structure & Algorithm

CHAPTER 1 BASIC TERMINOLOGY: ELEMENTARY DATA ORGANIZATION

Introduction

Data Structures are the programmatic way of storing data so that data can be used efficiently.

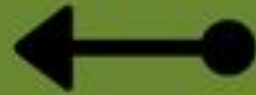


= PROGRAMS!

ALGORITHM



**DATA
STRUCTURE**



Why Learn Data Structure and Algorithms?

- ▶ As applications are getting complex and data rich, there are three common problems that applications face now-a-days.
 - ❑ Data Search
 - ❑ Processor Speed
 - ❑ Multiple Request

Why to Learn Data Structure and Algorithms?



- ❑ **Data Search** – Consider an inventory of 1 million(10^6) items of a store. If the application is to search an item, it has to search an item in 1 million(10^6) items every time slowing down the search. As data grows, search will become slower.
- ❑ **Processor speed** – Processor speed although being very high, falls limited if the data grows to billion records.
- ❑ **Multiple requests** – As thousands of users can search data simultaneously on a web server, even the fast server fails while searching the data.

Why Data Structure and Algorithms?

To solve the above-mentioned problems, data structures come to rescue.

Data can be organized in a data structure in such a way that all items may not be required to be searched, and the required data can be searched almost instantly.

Elementary Data Organization

Outline

- ▶ Data, Entity and Information (basic terminology used in data structure)
- ▶ Primitive data types
- ▶ Non primitive data Types
- ▶ Data structure
 - ▶ Definition
 - ▶ Classification
- ▶ Data structure operations

Basic terminology used in data structure



- ▶ Data
- ▶ Data Item
- ▶ Group Items
- ▶ Elementary Items
- ▶ Attribute & Entity
- ▶ Entity Set
- ▶ Field
- ▶ Record
- ▶ File

Basic Terminology

- ▶ **Data** – Data are values or set of values.
- ▶ **Data Item** – Data item refers to single unit of values.
- ▶ **Group Items** – Data items that are divided into sub items are called as Group Items.
- ▶ **Elementary Items** – Data items that cannot be divided are called as Elementary Items.
- ▶ **Attribute and Entity** – An entity is that which contains certain attributes or properties, which may be assigned values.
- ▶ **Entity Set** – Entities of similar attributes form an entity set.

Basic Terminology

- ▶ **Field** – Field is a single elementary unit of information representing an attribute of an entity.
- ▶ **Record** – Record is a collection of field values of a given entity.
- ▶ **File** – File is a collection of records of the entities in a given entity set.

Primitive data types

- ▶ These are the data structures which are directly supported by the machine. i.e. Any operation can be performed in these data items.
- ▶ The different primitive data types are
 - ✓ Integer
 - ✓ Float
 - ✓ Double
 - ✓ Character
 - ✓ Boolean

Non Primitive data types

- ▶ These Data structures do not allow any specific instructions to be performed on the Data items directly.
- ▶ The different non primitive data types are
 - ✓ Arrays
 - ✓ Structures
 - ✓ Unions
 - ✓ Class etc.

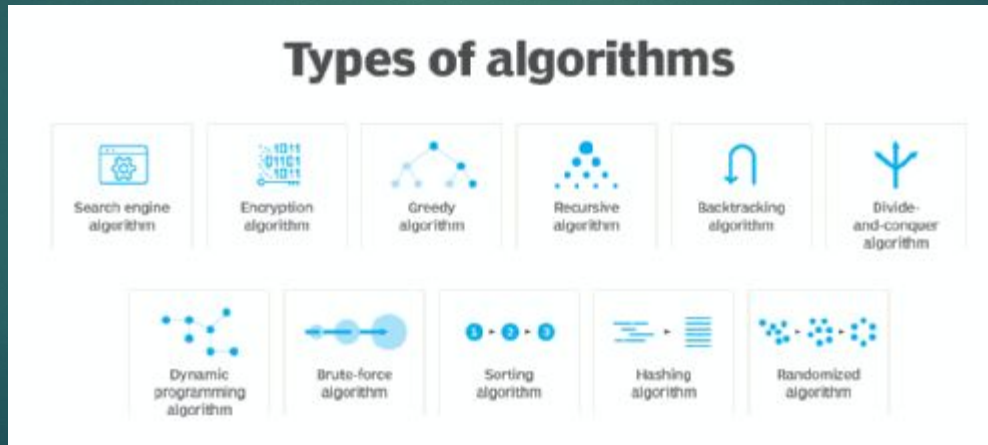
Data structure

- ▶ A *data structure* is an arrangement of data in a computer's memory or even disk storage.
- ▶ An example of several common data structures are arrays, linked lists, queues, stacks, binary trees, and hash tables.

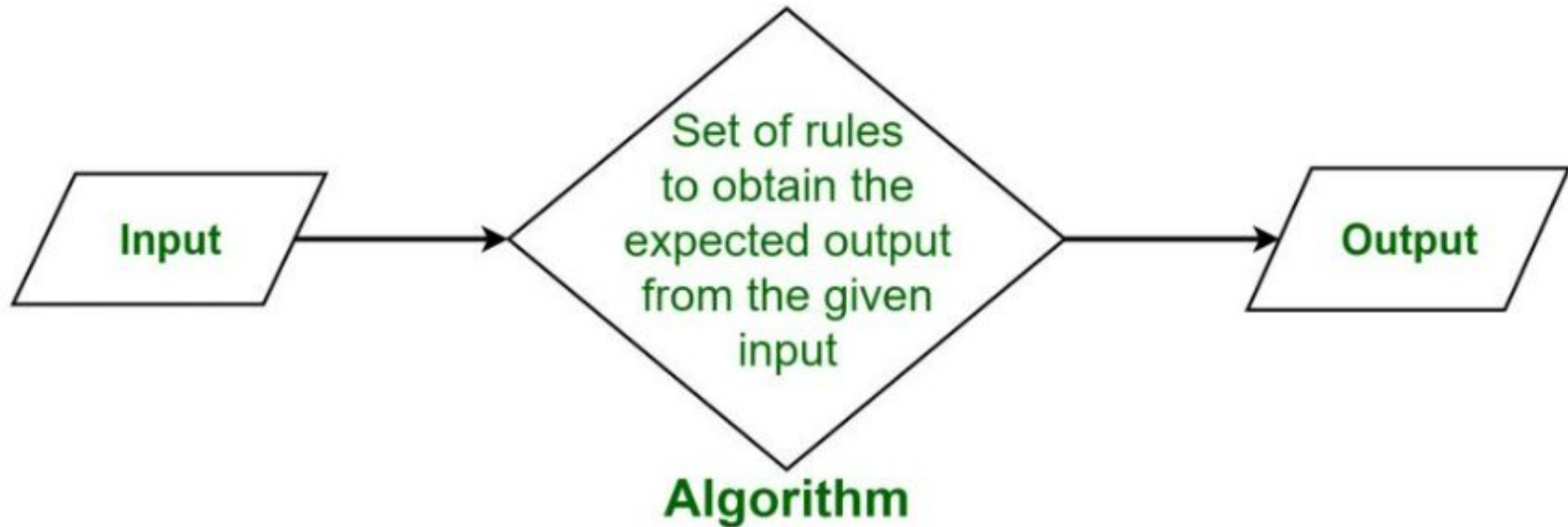


Data structure

- **Algorithms**, on the other hand, are used to manipulate the data contained in these data structures as in searching and sorting. Many algorithms apply directly to a specific data structures.



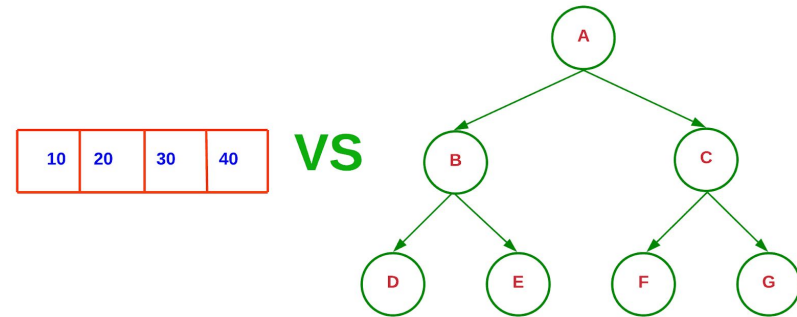
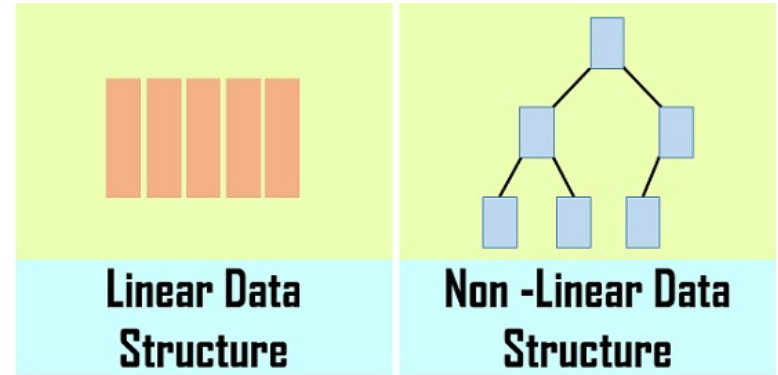
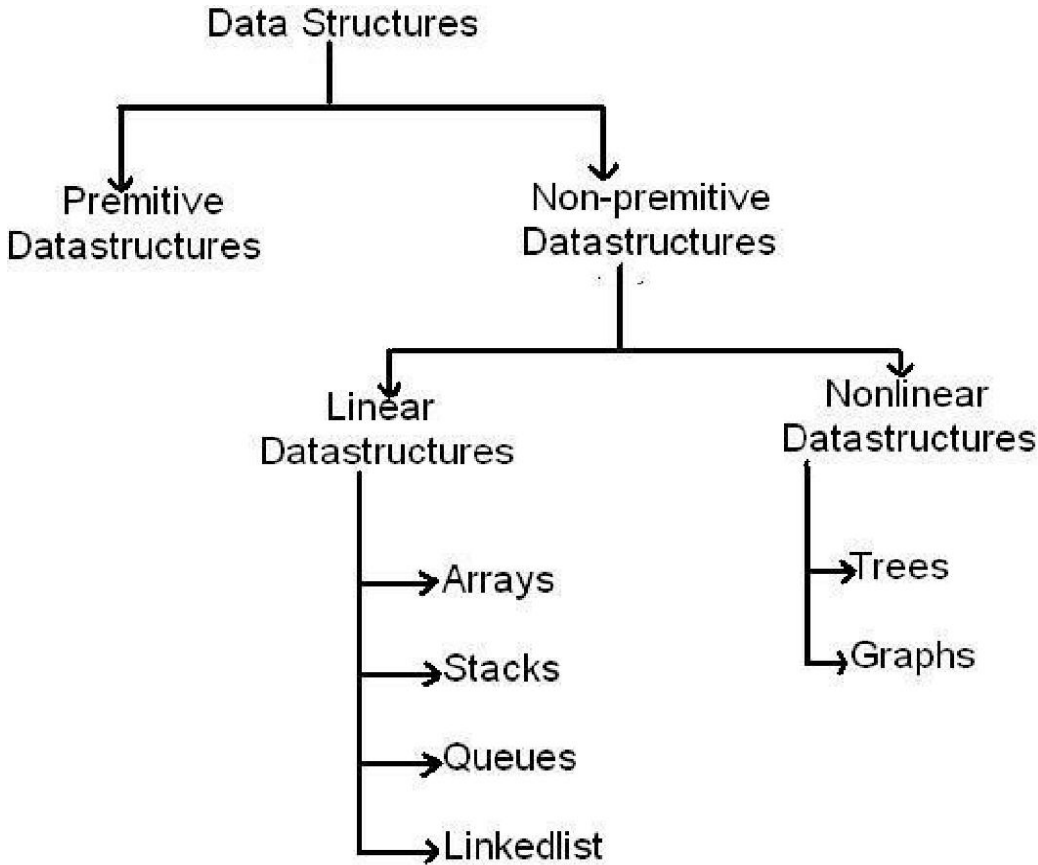
What is Algorithm?



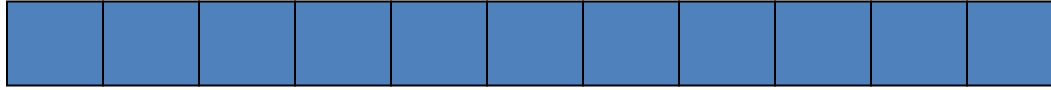
Classification

There are two types of data structure. They are

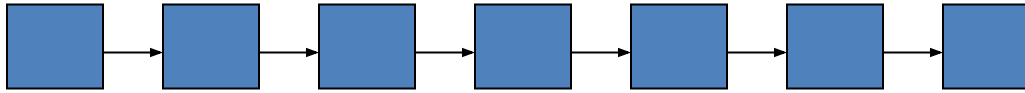
- ▶ Linear Data structures
- ▶ Non-Linear Data structures



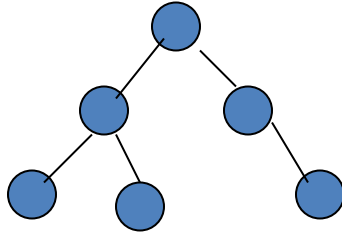
Types of data structures



Array



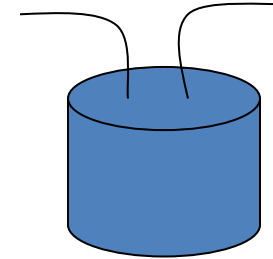
Linked List



Tree



Queue



Stack

There are many, but we named a few. We'll learn these data structures in great detail!

Linear Data structures



- ▶ This Data Structures involve arranging the elements in Linear fashion.
- ▶ Eg.
 - **Stacks**
 - **Queue**
 - **Lists**

Non-Linear Data structures

- ▶ This Data structures involve representing the elements in Hierarchical order.
- ▶ Eg:
 - **Trees**
 - **Graphs**

Data Structure Operations

- ▶ When working with certain data structures you need to know how to **insert** new data, **search** for a specified item, and **deleting** a specific item.

Data Structure Operations



- ❑ Traversing
 - Accessing each data element exactly once so that certain items in the data may be processed
- ❑ Searching
 - Finding the location of the data element (key) in the structure
- ❑ Insertion
 - Adding a new data element to the structure

Data Structure Operations (cont.)



- ❑ Deletion
 - Removing a data element from the structure
- ❑ Sorting
 - Arrange the data elements in a logical order (ascending/descending)
- ❑ Merging
 - Combining data elements from two or more data structures into one

The following computer problems can be solved using Data Structures

- ▶ Fibonacci number series(0,1,1,2,3,5....)
- ▶ Knapsack problem
- ▶ Tower of Hanoi
- ▶ All pair shortest path by Floyd-Warshall
- ▶ Shortest path by Dijkstra
- ▶ Project scheduling

What is algorithm?

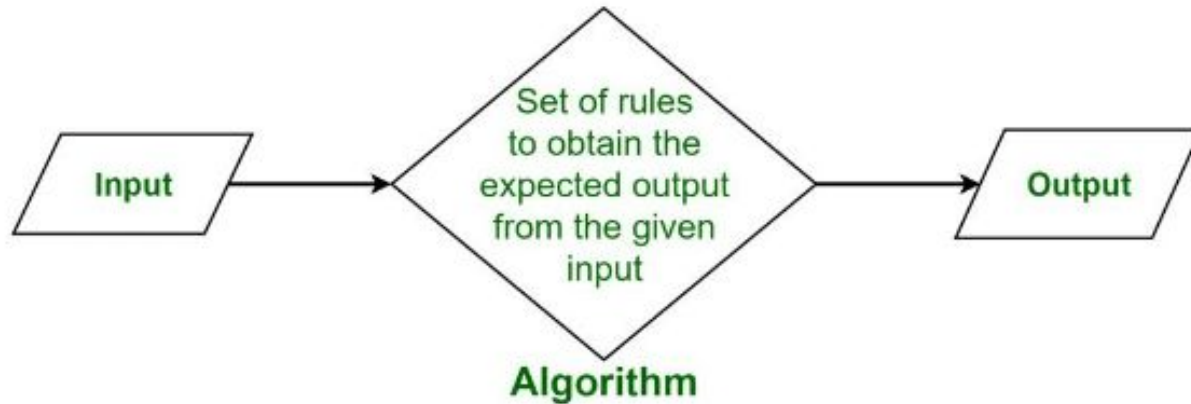


- ❑ A finite set of instructions which accomplish a particular task
- ❑ A method or process to solve a problem
- ❑ Transforms input of a problem to output

Algorithm = Input + Process + Output

Algorithm development is an art – it needs practice, practice and only practice!

What is Algorithm?



Characteristics of an Algorithm

- ▶ **Unambiguous** – Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
- ▶ **Input** – An algorithm should have 0 or more well-defined inputs.
- ▶ **Output** – An algorithm should have 1 or more well-defined outputs, and should match the desired output.
- ▶ **Finiteness** – Algorithms must terminate after a finite number of steps.
- ▶ **Feasibility** – Should be feasible with the available resources.
- ▶ **Independent** – An algorithm should have step-by-step directions, which should be independent of any programming code.

What is a good algorithm?



- ❑ It must be correct
- ❑ It must be finite (in terms of time and size)
- ❑ It must terminate
- ❑ It must be unambiguous
 - Which step is next?
- ❑ It must be space and time efficient

A program is an instance of an algorithm, written in some specific programming language

A simple algorithm



❑ Problem: Find maximum of a , b , c

❑ Algorithm

- Input = a , b , c
- Output = max
- Process

```
Let max = a
```

```
If b > max then
```

```
    max = b
```

```
If c > max then
```

```
    max = c
```

```
Display max
```

Order is very important!!!

Algorithm development: Basics



- ❑ Clearly identify:
 - what output is required?
 - what is the input?
 - What steps are required to transform input into output
 - The most crucial bit
 - Needs problem solving skills
 - A problem can be solved in many different ways
 - Which solution, amongst the different possible solutions is optimal?

How to express an algorithm?



- ❑ A sequence of steps to solve a problem
- ❑ We need a way to express this sequence of steps
 - Natural language (NL) is an obvious choice, but not a good choice. Why?
 - NLS are notoriously ambiguous (unclear)
 - Programming language (PL) is another choice, but again not a good choice. Why?
 - Algorithm should be PL independent
 - We need some balance
 - We need PL independence
 - We need clarity
 - Pseudo-code provides the right balance

What is pseudo-code?



- ❑ Pseudo-code is a short hand way of describing a computer program
- ❑ Rather than using the specific syntax of a computer language, more general wording is used

What is pseudo-code?

- ❑ It is a mixture of NL and PL expressions, in a systematic way
- ❑ Using pseudo-code, it is easier for a non-programmer to understand the general workings of the program

Pseudo-code: general guidelines

- ❑ Use PLs construct that are consistent with modern high level languages, e.g. C++, Java, ...
- ❑ Use appropriate comments for clarity
- ❑ Be simple and precise

Components of Pseudo-code

□ Expressions

- Standard mathematical symbols are used
 - Left arrow sign ($\leftarrow$) as the assignment operator in assignment statements (equivalent to the = operator in Java)
 - Equal sign (=) as the equality relation in Boolean expressions (equivalent to the "==" relation in Java)
 - For example
 - Sum $\leftarrow$ 0
 - Sum $\leftarrow$ Sum + 5

What is the final value of sum?

Components of Pseudo-code (cont.)

❑ **Decision structures** (if-then-else logic)

- **if** condition **then** true-actions [**else** false-actions]
- We use indentation to indicate what actions should be included in the true-actions and false-actions
- For example

```
if marks > 50 then  
    print "Congratulation, you are passed!"  
else  
    print "Sorry, you are failed!"  
end if
```

What will be the output if marks are equal to 75?

Components of Pseudo-code (cont.)

□ Loops (Repetition)

▪ **Pre-condition loops**

○ While loops

- **while** condition **do** actions
- We use indentation to indicate what actions should be included in the loop actions
- For example

```
while counter < 5 do  
    print "Welcome to CS204!"  
    counter ← counter + 1  
end while
```

What will be the output if counter is initialised to 0, 7?

Components of Pseudo-code (cont.)

□ Loops (Repetition)

▪ **Pre-condition loops**

○ For loops

- **for** variable-increment-definition **do** actions
- For example

```
for counter  $\leftarrow$  0; counter < 5; counter  $\leftarrow$  counter + 2 do  
    print “Welcome to CS204!”  
end for
```

What will be the output?

Components of Pseudo-code (cont.)

□ Loops (Repetition)

▪ **Post-condition loops**

○ Do loops

- **do** actions **while** condition
- For example

do

 print “Welcome to CS204!”

 counter $\leftarrow$ counter + 1

while counter < 5

What will be the output, if counter was initialised to 10?

The body of a post-condition loop must execute at least once

Components of Pseudo-code (cont.)

❑ Method declarations

- Return_type method_name (parameter_list) method_body
- For example

```
integer sum ( integer num1, integer num2)
start
    result ← num1 + num2
end
```

❑ Method calls

- object.method (args)
- For example

```
mycalculator.sum(num1, num2)
```

Components of Pseudo-code (cont.)

❑ Method returns

- **return** value

- For example

integer *sum* (integer num1, integer num2)

start

 result $\leftarrow$ num1 + num2

return result

end

Components of Pseudo-code (cont.)

□ Comments

- `/* Multiple line comments go here. */`
- `// Single line comments go here`
- Some people prefer braces `{}`, for comments

□ Arrays

- $A[i]$ represents the i th cell in the array A .
- The cells of an n -celled array A are indexed from $A[0]$ to $A[n - 1]$ (consistent with Java).

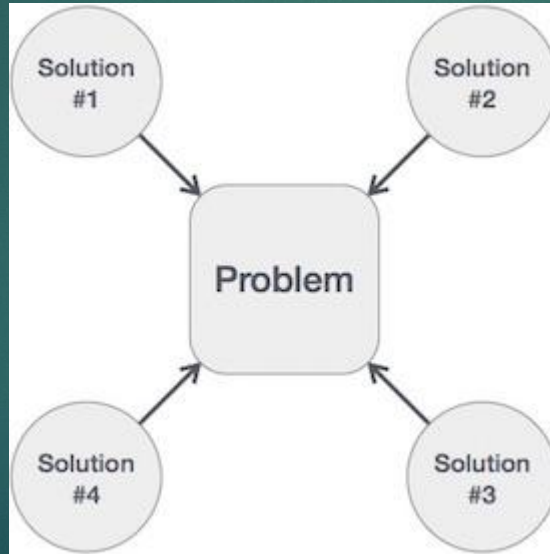
How to Write an Algorithm?



- ▶ There are no well-defined standards for writing algorithms.
- ▶ Rather, it is problem and resource dependent. Algorithms are never written to support a particular programming code.

How to Write an Algorithm?

- We design an algorithm to get a solution of a given problem. A problem can be solved in more than one ways.



Let's try to learn algorithm-writing by using an example.

Problem – Design an algorithm to add two numbers and display the result.

Step 1 – START

Step 2 – declare three integers **a**, **b** & **c**

Step 3 – define values of **a** & **b**

Step 4 – add values of **a** & **b**

Step 5 – store output of step 4 to **c**

Step 6 – print **c**

Step 7 – STOP

Algorithms tell the programmers how to code the program.

Alternatively, the algorithm can be written as –

Step 1 – START ADD

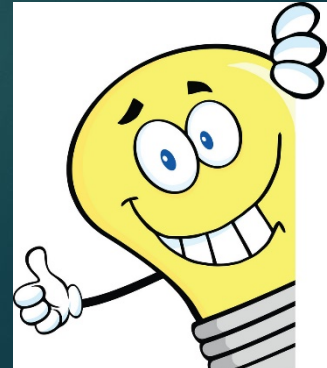
Step 2 – get values of **a & **b****

Step 3 – $c \leftarrow a + b$

Step 4 – display c

Step 5 – STOP

- ✓ In design and analysis of algorithms, usually the second method is used to describe an algorithm.
- ✓ It makes it easy for the analyst to analyze the algorithm ignoring all unwanted definitions.





END OF CHAPTER 1